

Inyección de dependencias y estereotipos

Java con Spring Boot de 0 a Pro

Fundamentos de Spring Boot • Video #4



¿Qué aprenderás hoy?

1 Comprender la inyección de dependencias

Entender qué es DI y por qué es fundamental en Spring Boot para crear aplicaciones bien estructuradas.

3 Aplicar estereotipos correctamente

Usar `@Component`, `@Service`, `@Repository` y `@Controller` para organizar tu arquitectura.

2 Dominar las formas de inyección

Conocer constructor, setter y field injection, y cuál es la práctica recomendada.

4 Construir una arquitectura limpia

Conectar beans de forma desacoplada y mantenible usando patrones profesionales.

Agenda del video

01

El problema del acoplamiento

Por qué crear objetos manualmente genera problemas

03

Formas de inyección

Constructor, setter y field injection

05

Demo práctica

Conectando beans en una aplicación real

02

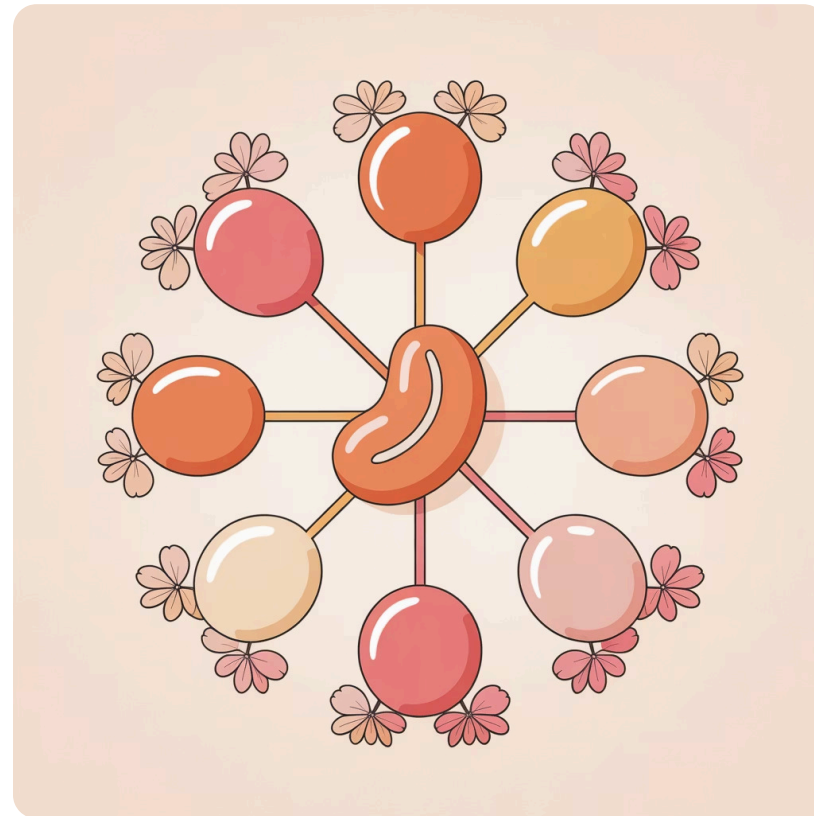
Qué es la inyección de dependencias

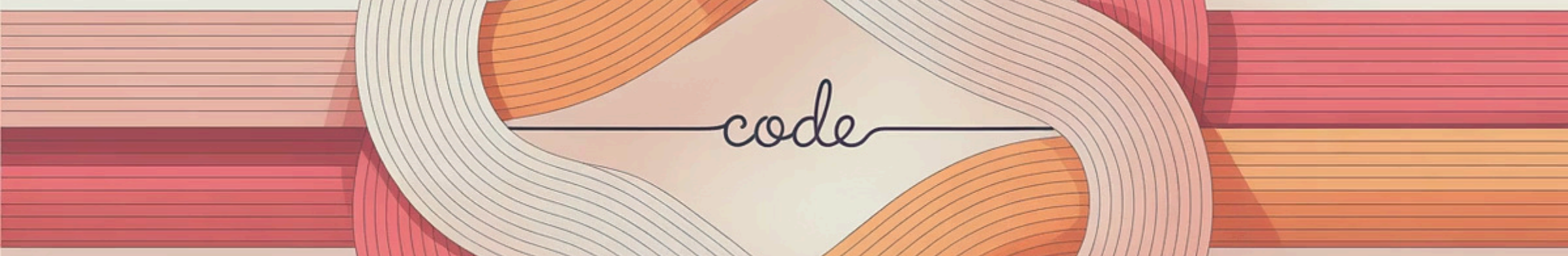
La solución elegante que ofrece Spring

04

Estereotipos en acción

@Service, @Repository, @Controller y más





code

El problema sin inyección de dependencias

Acoplamiento fuerte = código frágil

Cuando instanciamos objetos manualmente con `new`, creamos dependencias rígidas que son difíciles de mantener, probar y reutilizar.

Código difícil de testear

No puedes sustituir dependencias por mocks fácilmente

Cambios en cascada

Modificar una clase requiere tocar múltiples archivos

Duplicación de lógica

Instancias repetidas del mismo objeto en diferentes lugares

¿Qué es la inyección de dependencias?

Delegar

La inyección de dependencias (DI) es un patrón donde **delegamos a Spring** la responsabilidad de crear y conectar los objetos que nuestra aplicación necesita.

En lugar de usar `new`, le decimos a Spring: *"tú encárgate de darme lo que necesito"*.



- Inversión de control:** No controlamos cuándo ni cómo se crean los objetos. Spring lo hace por nosotros.

Beneficios de la inyección de dependencias



Desacoplamiento

Las clases no dependen de implementaciones concretas, sino de abstracciones



Reutilización

Los beans pueden ser inyectados en múltiples lugares sin duplicar código



Testabilidad

Fácil sustitución por mocks y stubs en pruebas unitarias



Configurabilidad

Cambiar comportamientos sin modificar el código fuente

Formas de inyección en Spring



Constructor

Dependencias se pasan al crear el objeto

```
@Autowired
public UserService(UserRepository repo)
{
    this.repository = repo;
}
```



Setter

Dependencias se asignan después de la creación

```
@Autowired
public void
setRepository(UserRepository repo) {
    this.repository = repo;
}
```



Field

Inyección directa en el campo

```
@Autowired
private UserRepository repository;
```

¡Usa constructor!

¿Por qué preferir inyección por constructor?

- **Inmutabilidad:** Los campos pueden ser final
- **Dependencias obligatorias:** No hay objetos en estado inválido
- **Fácil testing:** Constructor explícito para tests
- **Detección temprana:** Fallos al crear el contexto, no en runtime

⚠ **Evita field injection** en código de producción. Es difícil de testear.



¿Qué son los estereotipos?



Los **estereotipos** son anotaciones que especializan `@Component`, indicando el **rol específico** que cumple cada bean en tu aplicación.

Son como *etiquetas semánticas* que organizan tu código según responsabilidades:

- Facilitan la comprensión del código
- Permiten configuraciones específicas
- Mejoran la mantenibilidad
- Habilitan funcionalidades adicionales

Los cuatro estereotipos principales



@Component

Propósito general: Cualquier clase que debe ser gestionada por Spring

Úsalo cuando no encaje en otros estereotipos específicos



@Service

Lógica de negocio: Clases que contienen las reglas y procesos del dominio

Ideal para servicios, calculadoras, procesadores



@Repository

Acceso a datos: Clases que interactúan con bases de datos o sistemas externos

Incluye manejo automático de excepciones de persistencia

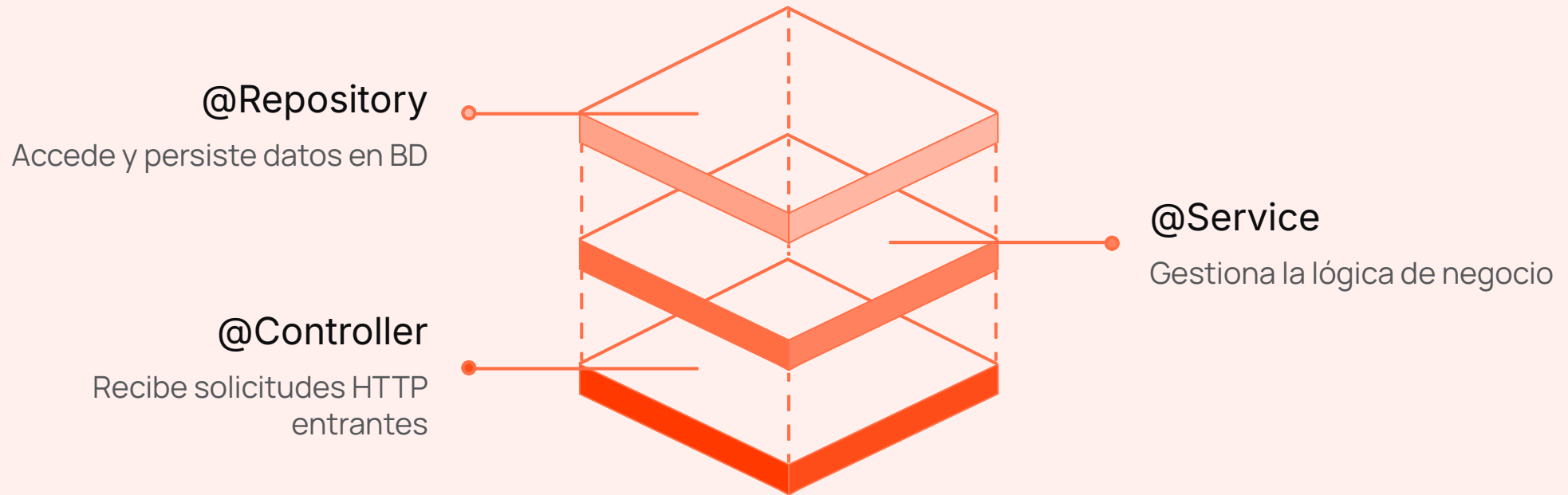


@Controller

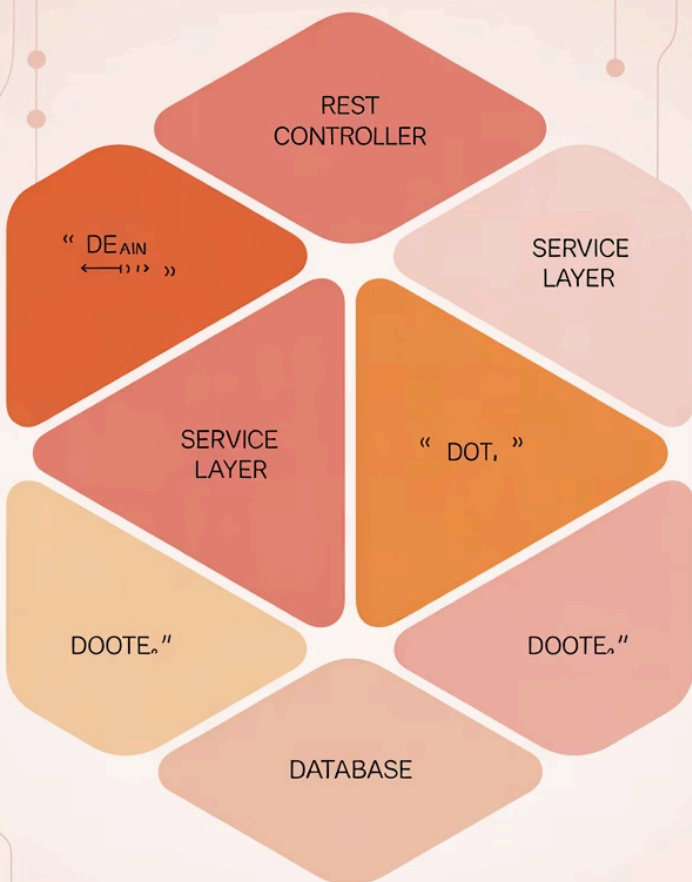
Capa web: Clases que manejan peticiones HTTP y coordinan respuestas

Punto de entrada para APIs REST y aplicaciones web

Arquitectura con estereotipos



Cada estereotipo tiene su lugar específico en la arquitectura, creando **separación clara de responsabilidades** y facilitando el mantenimiento del código.



Spring Boot

Application Architecture

Resumen de conceptos clave

Inyección de dependencias

Spring gestiona la creación y conexión de beans automáticamente

Constructor injection

La forma recomendada: inmutable, testeable y segura

Estereotipos semánticos

@Service, @Repository, @Controller organizan tu código por responsabilidades



Arquitectura limpia conseguida: Código desacoplado, testeable y fácil de mantener.



Demo

Conectando beans en la práctica

[Momento del código en vivo]

Vamos a crear un UserController que usa UserService, que a su vez depende de UserRepository, todo conectado mediante inyección por constructor.



@Controller

Maneja peticiones HTTP



@Service

Procesa lógica de negocio



@Repository

Accede a los datos

Próximos pasos en tu aprendizaje

¿Qué viene después?

En el próximo video profundizaremos en la **configuración de propiedades** con Spring Boot:

- application.properties vs application.yml
- Perfiles de configuración (dev, prod, test)
- @ConfigurationProperties para mapear valores
- Variables de entorno y configuración externa

Aprenderás a hacer tu aplicación **configurable y adaptable** a diferentes entornos.



Practica lo aprendido: Crea tu propia aplicación con los tres estereotipos conectados.



¡Seguí creciendo!

Ya dominás la inyección de dependencias y estereotipos.

¡El siguiente nivel te espera!

Java con Spring Boot de 0 a Pro

Fundamentos de Spring Boot • Video #4